Session 1: LLM Fundamentals + First Streaming API Call

Duration	2 hours
Split	Part 1: Mental model (30 min reading) | Part 2: Build it (90 min coding)
Tool	Groq -- free, no credit card -- console.groq.com
Goal	Understand how LLMs work at the intuition level and stream your first token to a terminal

PART 1 — The Mental Model

Read this section before writing a single line of code. The five concepts below underpin every architectural decision you will make as an AI engineer. They take 30 minutes to read and will save you days of confusion later.

---

1. How a transformer predicts the next token

Imagine you are playing a word-association game where instead of one word, you have the entire history of the internet in your head.

Someone gives you: *"The cat sat on the..."*

Your brain does not randomly pick the next word. It weighs options — "mat" is likely, "moon" is possible, "quantum" is very unlikely. The weight you assign to each option comes from every time you have seen similar sentence patterns before.

A transformer does exactly this, but at the scale of billions of examples, and it does it for **tokens**, not words.

The three things it actually does

1. It looks at every previous token simultaneously, not one at a time.
This is the key insight that made transformers better than everything before them. Older models read left to right like you are reading this sentence. Transformers look at all tokens at once and ask: for each token, which other tokens in this sequence are most relevant to understanding it? The word 'it' in that last sentence becomes meaningful only when connected back to 'transformer' — a transformer figures that connection out automatically.

2. It assigns attention weights.
When predicting the next token after 'The cat sat on the', the model pays high attention to 'cat' and 'sat' and very low attention to 'The'. This is called the **attention mechanism**. You do not need to know the math — just know that this is what the word 'attention' means when you read it in any AI article.

3. It samples from a probability distribution.
After all the attention calculations, the model produces a probability for every token in its vocabulary — roughly 100,000 tokens. 'mat' might get 34%, 'floor' gets 18%, 'roof' gets 4%, and so on. Temperature controls how you sample from this distribution. Temperature 0 always picks the highest-probability token. Temperature 1 samples proportionally — sometimes picking the second or third option, which is where creativity comes from.

> **NOTE**
> Why this matters for your work as a backend engineer:
> - This is why the model sometimes confidently says something wrong. Wrong tokens can still have non-zero probability and get sampled.
> - This is why longer prompts with more context produce better answers. More tokens to attend to means better probability estimates.
> - This is why you will hit weird failures at the edge of the context window. When the sequence gets very long, attention across 200,000 tokens becomes harder to maintain coherently.
> You do not need to understand matrix multiplication or self-attention heads. The mental model is: probability distribution over all possible next tokens, shaped by everything that came before. That is enough to make good architectural decisions.

---

2. What is a token

A token is not a word. It is a chunk of text — roughly 3 to 4 characters on average.

Text	Tokens	Count
"hamburger"	"ham" + "bur" + "ger"	3
"Hello"	"Hello"	1
"getUserById"	"get" + "User" + "By" + "Id"	4
1 page of English text	~500 tokens	~500
This entire PDF	~8,000 tokens	~8,000

Why this matters to you as a backend engineer: every API call has a token limit. If your NestJS service sends a 50-page document to Claude, you will hit the context window and the call will fail. You need to think in tokens the way you think in bytes when sizing a payload. This is exactly why Session 4 (RAG) spends so much time on chunking strategy — it is all token arithmetic.

---

3. What is a context window

The context window is RAM for the model. Everything the model can see in one call — your system prompt, the conversation history, the document you pasted, the answer it is generating — all of it must fit inside the context window.

Model	Context window	Rough equivalent

Claude Sonnet 4	200,000 tokens	~150,000 words / ~500 pages
GPT-4o	128,000 tokens	~96,000 words / ~320 pages
Llama 3.3 70B (Groq)	128,000 tokens	~96,000 words / ~320 pages

When you build RAG in Session 4, chunking documents into small pieces exists entirely because of this constraint. A 500-page report cannot fit in one call, so you cut it into 300-token chunks, embed them, and retrieve only the 5 most relevant chunks at query time. The context window is why RAG exists.

---

4. What is temperature

Temperature is a number between 0 and 1 (sometimes up to 2) that controls how the model samples from its probability distribution.

Temperature	Behaviour	Use it for
0	Always picks the highest-probability token. Deterministic — same input always gives same output.	JSON extraction, classification, structured outputs, SQL generation
0.3 – 0.5	Mostly deterministic with slight variation. Good balance for most backend tasks.	Summarisation, Q&A, most production API features
0.7 – 1.0	Samples freely. Creative but less predictable. Same prompt gives different answers each time.	Creative writing, brainstorming, chat personas

Rule of thumb for backend work: always start at temperature 0. Only raise it if you have a specific reason to want variation. A classification endpoint that sometimes gives a different category on the same input is a bug, not a feature.

---

5. What are model tiers

Every provider offers three rough tiers. Picking the wrong tier is the most common and most expensive mistake in production AI engineering.

Tier	Examples	Cost	Speed	Best for
Small / fast	Claude Haiku GPT-4o-mini Llama 3.1 8B	~$0.10 /M tokens	Fast	Classification, routing, simple extraction, high-volume tasks
Mid (workhorse)	Claude Sonnet GPT-4o Llama 3.3 70B	~$3 /M tokens	Moderate	90% of production features. RAG answers, summarisation, chat, code generation
Large / frontier	Claude Opus o1 / o3 Llama 3.1 405B	~$15+ /M tokens	Slow	Complex multi-step reasoning, research tasks, when mid-tier fails

Daily decision rule: always start with mid-tier in development. Once it works, test the small model. If quality holds — ship the small model and save 30x on cost. If quality drops — keep mid-tier. Only escalate to large when mid-tier genuinely fails on your specific task.

PART 2 — Build It (90 minutes)

Four progressively deeper coding steps. Run each one, observe what it prints, then read the 'Observe' note before moving to the next.

Setup (5 minutes)

Sign up at **console.groq.com** and create a free API key. Then run:

terminal

```
mkdir ai-session-1 && cd ai-session-1
npm init -y
npm install groq-sdk dotenv
```

Create a **.env** file in the project root:

.env

```
GROQ_API_KEY=your_key_here
```

1 Basic completion (15 minutes)

Create **basic.js** and run it with **node basic.js**

basic.js

```
import Groq from "groq-sdk";
import * as dotenv from "dotenv";
dotenv.config();

const client = new Groq({ apiKey: process.env.GROQ_API_KEY });

const response = await client.chat.completions.create({
  model: "llama-3.3-70b-versatile",
  messages: [
    { role: "user", content: "What is a token in an LLM? Answer in 3 sentences." }
  ],
  temperature: 0,
  max_tokens: 200,
});

console.log(response.choices[0].message.content);
console.log("\n--- Usage ---");
console.log("Prompt tokens:     ", response.usage.prompt_tokens);
console.log("Completion tokens: ", response.usage.completion_tokens);
console.log("Total tokens:      ", response.usage.total_tokens);
```

OBSERVE

OBSERVE: Look at the prompt token count. Your short question uses more tokens than you might expect because every API call includes overhead before your message even starts. This is the kind of thing that quietly inflates costs in production when you are calling the API thousands of times per day.

2 Streaming (20 minutes)

Create **streaming.js** and run it with **node streaming.js**

streaming.js

```
import Groq from "groq-sdk";
import * as dotenv from "dotenv";
dotenv.config();

const client = new Groq({ apiKey: process.env.GROQ_API_KEY });

console.log("Streaming response:\n");

const stream = await client.chat.completions.stream({
  model: "llama-3.3-70b-versatile",
  messages: [
    {
      role: "user",
      content: "Explain how a transformer predicts the next token. Use a simple analogy."
    }
  ],
  temperature: 0.7,
  max_tokens: 300,
});

for await (const chunk of stream) {
  const token = chunk.choices[0]?.delta?.content || "";
  process.stdout.write(token);
}

console.log("\n\nDone.");
```

OBSERVE

OBSERVE: Tokens print one by one as they are generated. This is exactly what powers the typing effect in ChatGPT. In Session 5 you will pipe this stream to a React UI.
For now, understand that streaming is not optional for good UX — a 3-second wait for a full response feels broken; the same 3 seconds with tokens appearing immediately feels fast. The model generates one token at a time anyway. Streaming just surfaces that natural cadence to the client.

3 System prompts and temperature (25 minutes)

Create **system-prompt.js** and run it with **node system-prompt.js**

system-prompt.js

```
import Groq from "groq-sdk";
import * as dotenv from "dotenv";
dotenv.config();

const client = new Groq({ apiKey: process.env.GROQ_API_KEY });

async function ask(systemPrompt, userMessage, temperature) {
  const response = await client.chat.completions.create({
    model: "llama-3.3-70b-versatile",
    messages: [
      { role: "system", content: systemPrompt },
      { role: "user",   content: userMessage  }
    ],
    temperature,
    max_tokens: 150,
  });
  return response.choices[0].message.content;
}

const userQ = "Should I use PostgreSQL or MongoDB for my app?";

const results = await Promise.all([
  ask("You are a helpful assistant.", userQ, 0),
  ask(
    "You are a terse senior engineer. Answer in one sentence, no fluff.",
    userQ, 0
  ),
  ask("You are a helpful assistant.", userQ, 1),
]);

console.log("=== Default system prompt, temp 0 ===");
console.log(results[0]);

console.log("\n=== Terse engineer system prompt, temp 0 ===");
console.log(results[1]);

console.log("\n=== Default system prompt, temp 1 ===");
console.log(results[2]);
```

OBSERVE

OBSERVE: The same question, three different responses. The system prompt shapes the persona and output style completely. Temperature 1 vs 0 changes the variation on the same prompt. This is the primary lever you will use in Session 2. You are seeing it here in raw form before we put structure around it.

4 Model tiers side by side (30 minutes)

Create **model-tiers.js** and run it with **node model-tiers.js**

model-tiers.js

```
import Groq from "groq-sdk";
import * as dotenv from "dotenv";
dotenv.config();

const client = new Groq({ apiKey: process.env.GROQ_API_KEY });

async function timedCall(model, prompt) {
  const start = Date.now();
  const response = await client.chat.completions.create({
    model,
    messages: [{ role: "user", content: prompt }],
    temperature: 0,
    max_tokens: 200,
  });
  return {
    model,
    answer: response.choices[0].message.content,
    tokens: response.usage.total_tokens,
    ms:     Date.now() - start,
  };
}

const prompt = `
Extract the following fields from this text as JSON:
- name
- amount
- currency
- date

Text: "John Doe transferred 4500 rupees to vendor account on 14th March 2025."
Return only valid JSON, nothing else.
`;

const [small, mid] = await Promise.all([
  timedCall("llama-3.1-8b-instant",    prompt),
  timedCall("llama-3.3-70b-versatile", prompt),
]);

console.log("=== Small model (8B) ===");
console.log("Answer:", small.answer);
console.log(`Time: ${small.ms}ms | Tokens: ${small.tokens}`);

console.log("\n=== Mid model (70B) ===");
console.log("Answer:", mid.answer);
console.log(`Time: ${mid.ms}ms | Tokens: ${mid.tokens}`);

console.log("\n--- Decision ---");
console.log("If both outputs are correct JSON -> use small model (30x cheaper).");
console.log("If small model hallucinated a field -> you need mid-tier.");
```

OBSERVE

OBSERVE: For simple structured extraction, the 8B model is often as accurate as the 70B but significantly faster. This is the model routing decision you will make in Session 7 when optimising production costs. The cost difference is not small — at scale a 30x cost difference between small and mid-tier models is the difference between a feature being economically viable or not.

Session 1 — Review Questions

Answer each question out loud or write it down before checking. When you are ready, bring your answers back to the chat window for review. Do not look back at the PDF while answering — the goal is retrieval under mild pressure, which is closer to what an interview feels like.

1. Context window and chunking

What is a context window, and why does its existence force you to chunk documents before storing them in a RAG system? Answer in 2 to 3 sentences without using the word 'limit'.

2. Temperature for a classification endpoint

You are building a NestJS endpoint that classifies incoming customer support tickets into one of five categories. What temperature do you set and why? What would happen if you used temperature 1 instead?

3. Streaming and HTTP

Your streaming.js file printed tokens one by one to the terminal. What has to be fundamentally different about the HTTP response your server sends when streaming vs when returning a normal JSON payload? Why can a standard res.json() call not stream tokens?

4. Model tier decision

A colleague says: just use the biggest model, it is more accurate. Give two specific situations where this advice is wrong, and explain the real decision framework you would use instead.

5. Connecting the dots

In model-tiers.js, both the 8B and 70B models were given the same extraction task. Explain why the 8B model might occasionally extract a wrong field value when the 70B does not, using what you know about how transformers predict tokens and probability distributions.

Once you have answered all five, paste your answers into the chat. Correct answers unlock Session 2: Prompt Engineering — where you will take the system prompt from Step 3 above and build it into a reliable, production-grade structured output template.